Academia\FrontEnd\JS\Academia.js

```
/*
Academia - live frontend client (no hardcoded data, no offline fallback)

Browser -> C server (port 3000) -> SQLite database.
Every value shown in the Student, Parent and Staff portals comes from:
  GET /api/student-data | /api/staff-data | /api/parent-data   (header X-Auth-Token)
Staff writes go to:
  POST /api/staff/grade | /api/staff/attendance               (see BackEnd_Patch/)
The data endpoints return JavaScript (const xState = {...};), not JSON, so
they are read as text and evaluated.
*/

var TOKEN_KEY = "academiaToken";
var USER_KEY = "academiaUser";

/* ---------- helpers ---------- */
function esc(v) {
    return String(v == null ? "" : v).replace(/[&<>"']/g, function (c) {
        return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }[c];
    });
}
function $(id) { return document.getElementById(id); }
function setHtml(id, html) { var el = $(id); if (el) el.innerHTML = html; }
function setText(id, txt) { var el = $(id); if (el) el.innerText = txt; }

function showError(msg) {
    var el = $("page-error");
    if (!el) { alert(msg); return; }
    el.style.display = "block";
    el.innerText = msg;
}

function table(headers, rows, emptyText) {
    var h = "<tr>" + headers.map(function (x) { return "<th>" + esc(x) + "</th>"; }).join("")
+ "</tr>";
    if (!rows.length) {
        return h + "<tr><td colspan='" + headers.length + "'>" + esc(emptyText) + "</td>
</tr>";
    }
    return h + rows.map(function (r) {
        return "<tr>" + r.map(function (c) { return "<td>" + esc(c) + "</td>"; }).join("") +
"</tr>";
    }).join("");
}
function pct(score) { return Number(score).toFixed(1) + "%"; }
function presentText(p) { return p ? "Present" : "Absent"; }

/* ---------- session ---------- */
```

```
function logout() {
    sessionStorage.removeItem(TOKEN_KEY);
    sessionStorage.removeItem(USER_KEY);
    window.location.href = "Academia_Auth.html";
}
function authHeaders(extra) {
    var h = extra || {};
    h["X-Auth-Token"] = sessionStorage.getItem(TOKEN_KEY) || "";
    return h;
}

/* Reads one of the C-generated state files and returns the state object. */
function loadState(path, varName) {
    return fetch(path, { headers: authHeaders() }).then(function (res) {
        if (res.status === 401) { logout(); throw new Error("Session expired. Please sign in again."); }
        if (res.status === 403) { throw new Error("This account is not allowed to view that page."); }
        if (!res.ok) { throw new Error("Server error (" + res.status + ")."); }
        return res.text();
    }).then(function (text) {
        var state = new Function(text + "\nreturn " + varName + ";")();
        if (!state || state.success !== true) { throw new Error("The database has no record for this account."); }
        return state;
    });
}

function postJson(path, body) {
    return fetch(path, {
        method: "POST",
        headers: authHeaders({ "Content-Type": "application/json" }),
        body: JSON.stringify(body)
    }).then(function (res) {
        if (res.status === 401) { logout(); throw new Error("Session expired."); }
        return res.json().catch(function () { return { success: false, message: "Server error (" + res.status + ")." }; });
    });
}

/* ---------- login (Academia_Auth.html) ---------- */
var ROLE_PAGE = { student: "Academia_Student.html", parent: "Academia_Parents.html", staff: "Academia_Staff.html" };

function goPortal() {
    var u = $("username"), p = $("password"), r = $("role"), err = $("err");
    if (!u || !p || !r) return;
    if (err) err.innerText = "";
    var username = u.value.trim(), password = p.value.trim(), role = r.value;
    if (username === "" || password === "") {
        if (err) err.innerText = "Please enter both username and password.";
```

```
        return;
    }
    fetch("/api/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username: username, password: password, role: role })
    }).then(function (res) { return res.json(); }).then(function (data) {
        if (!data.success) {
            if (err) err.innerText = data.message || "Invalid login credentials.";
            return;
        }
        sessionStorage.setItem(TOKEN_KEY, data.token);
        sessionStorage.setItem(USER_KEY, JSON.stringify(data.user));
        window.location.href = ROLE_PAGE[data.user.role] || "Academia_Auth.html";
    }).catch(function () {
        if (err) err.innerText = "Cannot reach the Academia server. Start it and open http://localhost:3000.";
    });
}

/* ---------- student portal ---------- */
function showPage(pageId) {
    var d = $("dashboard-page"), c = $("courses-page");
    if (!d || !c) return;
    d.style.display = pageId === "dashboard" ? "block" : "none";
    c.style.display = pageId === "courses" ? "block" : "none";
}

function initStudent() {
    loadState("/api/student-data", "studentState").then(function (s) {
        var st = s.student, full = st.firstName + " " + st.lastName;
        setText("welcome-name", "Welcome, " + full);
        setText("student-name", full);
        setText("student-id", st.userId);
        setText("student-grade", st.gradeLevel);
        setText("student-gpa", Number(st.gpa).toFixed(2) + " / 4.00");

        setHtml("grades-table", table(["Course Title", "Grade"],
            s.grades.map(function (g) { return [g.course, pct(g.score)]; }), "No grades recorded yet."));
        setHtml("attendance-table", table(["Course Title", "Status"],
            s.attendance.map(function (a) { return [a.course, presentText(a.present)]; }), "No attendance recorded yet."));
        setHtml("assignments-table", table(["Course", "Assignment", "Due"],
            s.assignments.map(function (a) { return [a.course, a.assignmentName, a.dueDate]; }), "No assignments."));
        setHtml("courses-list", s.courses.length ? s.courses.map(function (c) {
            return "<div class='course-card'><h3>" + esc(c.course) + "</h3><p>" +
                   (c.status ? "Enrolled" : "Not active") + "</p></div><br>";
        }).join("") : "<p>You are not enrolled in any course.</p>");
        setHtml("announcements", s.announcements.length ? s.announcements.map(function (a) {
```

```
            return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p>" +
esc(a.message) +
                   "</p><small>" + esc(a.date) + "</small></div><br>";
        }).join("") : "<p>No announcements.</p>");
    }).catch(function (e) { showError(e.message); });
}

/* ---------- parent portal ---------- */
function showParentTab(tabId) {
    var o = $("parent-overview"), a = $("parent-announcements");
    if (!o || !a) return;
    o.style.display = tabId === "overview" ? "block" : "none";
    a.style.display = tabId === "announcements" ? "block" : "none";
}

function initParent() {
    loadState("/api/parent-data", "parentState").then(function (s) {
        setText("welcome-name", "Welcome, " + s.parent.firstName + " " + s.parent.lastName);
        setHtml("children-table", table(["Student", "Username", "Grade Level", "GPA"],
            s.children.map(function (c) { return [c.name, c.username, c.gradeLevel,
Number(c.gpa).toFixed(2)]; }),
            "No student is linked to this account."));
        setHtml("grades-table", table(["Student", "Course Title", "Recorded Score"],
            s.grades.map(function (g) { return [g.student, g.course, pct(g.score)]; }), "No
grades recorded yet."));
        setHtml("attendance-table", table(["Student", "Course Title", "Status"],
            s.attendance.map(function (a) { return [a.student, a.course,
presentText(a.present)]; }), "No attendance recorded yet."));
        setHtml("announcements", s.announcements.length ? s.announcements.map(function (a) {
            return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p>" +
esc(a.message) +
                   "</p><small>" + esc(a.date) + "</small></div><br>";
        }).join("") : "<p>No announcements.</p>");
    }).catch(function (e) { showError(e.message); });
}

/* ---------- staff portal ---------- */
var staffState = null;   // last data loaded from the server (declared in staff_states.js as
a const inside loadState, so no clash)

function showStaffTab(tabId) {
    ["management", "profile", "courses"].forEach(function (t) {
        var el = $("staff-" + t);
        if (el) el.style.display = t === tabId ? "block" : "none";
    });
}

function option(value, label) { return "<option value='" + esc(value) + "'>" + esc(label) + "
</option>"; }

function renderStaff(s) {
```

```
    staffState = s;
    var me = s.staff, full = me.firstName + " " + me.lastName;
    setText("staff-name", "Welcome, " + full);
    setText("staff-fullname", full);
    setText("staff-username", me.username);
    setText("staff-phone", me.phone);
    setText("staff-course-count", s.courses.length);

    setHtml("courses-table", table(["Course", "Active Students"],
        s.courses.map(function (c) { return [c.course, c.students]; }), "You are not assigned to any course."));

    /* unique students from the roster */
    var seen = {}, studentOpts = "";
    s.students.forEach(function (x) {
        if (!seen[x.studentId]) { seen[x.studentId] = 1; studentOpts += option(x.studentId, x.name); }
    });
    setText("staff-student-count", Object.keys(seen).length);
    var courseOpts = s.courses.map(function (c) { return option(c.courseId, c.course); }).join("");
    ["grade-student", "attendance-student"].forEach(function (id) { setHtml(id, studentOpts); });
    ["grade-course", "attendance-course"].forEach(function (id) { setHtml(id, courseOpts); });

    setHtml("grade-rows", s.students.length ? s.students.map(function (x) {
        return "<tr><td>" + esc(x.name) + "</td><td>" + esc(x.course) + "</td><td>" +
               (x.score === "" || x.score == null ? "-" : esc(pct(x.score))) + "</td></tr>";
    }).join("") : "<tr><td colspan='3'>No students enrolled in your courses.</td></tr>");

    setHtml("attendance-rows", s.attendance.length ? s.attendance.map(function (a) {
        return "<tr><td>" + esc(a.student) + "</td><td>" + esc(a.course) + "</td><td>" +
               presentText(a.present) + "</td></tr>";
    }).join("") : "<tr><td colspan='3'>No attendance recorded yet.</td></tr>");

    setHtml("announcements", s.announcements.length ? s.announcements.map(function (a) {
        return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p><b>" + esc(a.course) + ":</b> " +
               esc(a.message) + "</p><small>" + esc(a.date) + "</small></div><br>";
    }).join("") : "<p>No announcements for your courses.</p>");
}

function refreshStaff() {
    return loadState("/api/staff-data", "staffState").then(renderStaff);
}

function initStaff() {
    refreshStaff().catch(function (e) { showError(e.message); });
}

```

```
function saveGrade() {
    var score = $("grade-score").value;
    if (score === "" || isNaN(Number(score)) || Number(score) < 0 || Number(score) > 100) {
        showError("Enter a score between 0 and 100.");
        return;
    }
    postJson("/api/staff/grade", {
        studentId: $("grade-student").value,
        courseId: $("grade-course").value,
        score: String(score)
    }).then(function (res) {
        if (!res.success) throw new Error(res.message || "The server rejected the grade.");
        $("grade-score").value = "";
        return refreshStaff();
    }).catch(function (e) { showError(e.message); });
}

function saveAttendance() {
    postJson("/api/staff/attendance", {
        studentId: $("attendance-student").value,
        courseId: $("attendance-course").value,
        present: $("attendance-status").value
    }).then(function (res) {
        if (!res.success) throw new Error(res.message || "The server rejected the
attendance.");
        return refreshStaff();
    }).catch(function (e) { showError(e.message); });
}

/* ---------- start ---------- */
window.addEventListener("load", function () {
    var portal = document.body.getAttribute("data-portal");
    if (!portal) return;                        // login / landing pages
    if (!sessionStorage.getItem(TOKEN_KEY)) { window.location.href = "Academia_Auth.html";
return; }
    if (portal === "student") initStudent();
    else if (portal === "parent") initParent();
    else if (portal === "staff") initStaff();
});
```